

Mathematics and Statistics of Algorithmic Trading - Group Assignment

Department: Mathematics
Module: MATH0062
Academic Year: 2025-26

Group Members

Divij Wankhade - 25133033,
Max Batty - 22013004,
Syed Islam - 22139309,
Omar Qaalib - 25214443,



April 10, 2026

1 Introduction

The study considers two optimisation problems approached using deep learning techniques. Both problems, introduced in subsequent sections, are formulated under the Heston stochastic volatility model. This model serves as the foundation for generating simulated paths and represents an extension of the Black–Scholes model.

$$\frac{dS_t}{S_t} = \mu dt + \sqrt{Y_t} dW_t^1, \quad S_0 = s_0; \quad (1-1)$$

$$dY_t = \kappa(\theta - Y_t) dt + \eta\sqrt{Y_t} dW_t^2, \quad Y_0 = y_0. \quad (1-2)$$

Here S_t represents the price of the underlying stock price at time t and the variance process dY_t represents the instantaneous variance. The Brownian motions given by $(W_t^1, W_t^2)_{t \geq 0}$ are correlated and act as the randomness driving the model. The main difference between this model and more simpler ones is that the volatility is no longer constant and is instead a stochastic process in itself. In the case for the Heston model the volatility given by dY_t behaves like a Cox-Ingersoll-Ross process and exhibits mean reversion which aligns with the real world as high/low periods of volatility do not last forever.

Regarding portfolio assumptions, the investor holds a portfolio consisting of units of the underlying stock S_t and a risk-free asset Q_t with zero interest rate, i.e. $Q_t = 1 \quad \forall t$. Portfolio adjustments are assumed to occur at discrete daily intervals, consistent with a finite rebalancing frequency $\Delta t = 1$ day. The baseline parameters for the model are specified as:

$$s_0 = 1, \quad y_0 = 0.05, \quad \mu = 0.1, \quad \kappa = 1.2, \quad \theta = 0.05, \quad \eta = 0.1, \quad \rho = -0.5.$$

How varying some of these parameters affects our results will be addressed later on.

2 Portfolio Optimisation

2.1 Problem Statement

The objective of this section is to construct a portfolio allocation strategy that maximises the expected utility of the portfolio at a future time T , that is:

$$\sup_{\phi} \mathbb{E}[U(X_T)]$$

where X_T represents the value of our portfolio (consisting of the underlying and the risk-free asset) at the terminal time $T = 1$. The utility function used is given by $U(x) = -\exp(-\gamma x)$ with $\gamma = 1$. It has been predetermined that X_0 (initial value of the portfolio) is 1.

The first objective is to develop a deep learning based model capable of grasping the underlying dynamics of the problem and producing an effective trading strategy upon training. Subsequently, the dependence of the strategy on varying values of S_t and Y_t is examined. The second component of the problem introduces a terminal wealth constraint of the form $X_T \geq C = 0.75$ where $C > 0$ is a prescribed constant.

The incorporation of this constraint modifies the admissible set of strategies and requires a more conservative allocation policy. We subsequently investigate the impact of this constraint on the structure of the optimal strategy, with particular emphasis on how the requirement $\mathbb{P}(X_T \geq C) = 1$ affects the risk-taking behaviour of the investor.

2.2 Solution Methodology

The analysis begins with the construction of simulations based on the underlying framework described in the previous section. The corresponding implementation is provided in the accompanying Python files submitted with this report.

For the purposes of simulating the problem through time, trading can only occur during daily intervals over a horizon of $T=1$ years. Time is discretised accordingly below, where $N = 252$ comes from having 252 trading days in a year:

$$\Delta t = \frac{T}{N}, \quad N = 252 \quad t_n = n\Delta t, \quad n = 0, 1, \dots, N.$$

Note the switch from subscript t in continuous time to n for state values at the now discrete time points. We simulate the Heston model over 1 year using a daily grid. Since the variance process may become negative under a naive discretisation, we use a full-truncation modification and define $\tilde{Y}_n = \max(Y_n, 0)$ to stop square roots of negative values. Using the Milstein scheme with full truncation [1], we simulate the stock and variance processes by:

$$S_{n+1} = S_n + \mu S_n \Delta t + S_n \sqrt{\tilde{Y}_n} \Delta W_n^{(1)} + \frac{1}{2} S_n \sqrt{\tilde{Y}_n} \left(\frac{\eta \rho}{2} \right) ((\Delta W_n^{(1)})^2 - \Delta t),$$

$$Y_{n+1} = Y_n + \kappa(\theta - Y_n) \Delta t + \eta \sqrt{\tilde{Y}_n} \Delta W_n^{(2)} + \frac{1}{4} \eta^2 ((\Delta W_n^{(2)})^2 - \Delta t).$$

To generate correlated Brownian increments, we set:

$$\Delta W_n^{(1)} = \sqrt{\Delta t} Z_{1,n},$$

$$\Delta W_n^{(2)} = \sqrt{\Delta t} \left(\rho Z_{1,n} + \sqrt{1 - \rho^2} Z_{2,n} \right),$$

where $Z_{1,n}, Z_{2,n} \sim N(0, 1)$ independently.

The Milstein method is 2^{nd} order accurate within the volatility whereas the Euler-Maruyama discretisation method trialled is more computationally efficient and easier to implement. Hence, the Milstein method is preferable for path dependent problems. Since the discretisation scheme was reused for the path dependent Hedging and Pricing of an Exotic option problem in Section 3.2, the Milstein discretisation was adopted for the rest of this report. Figure 1 shows the simulated paths.

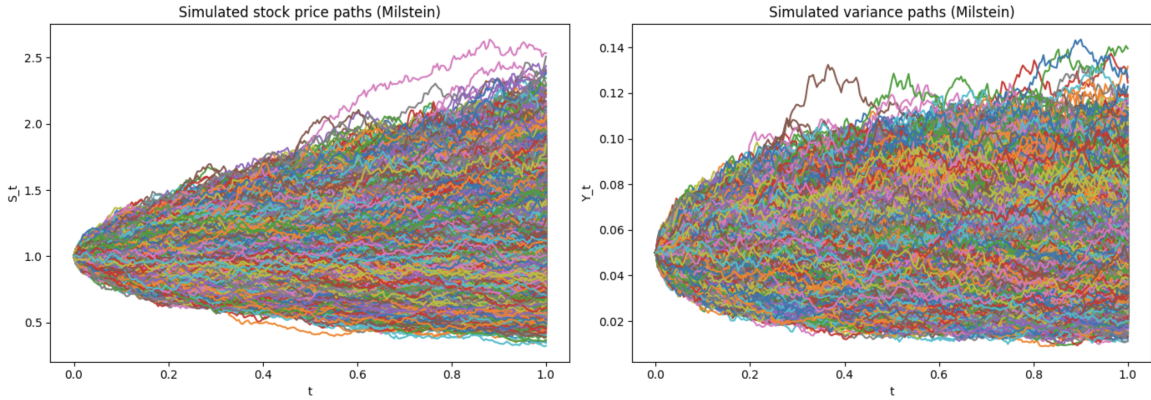


Figure 1: sample paths for the stock price (left) and variance (right)

A trading strategy $\phi = (\phi_n)_{n=0,1,\dots,N-1}$ is defined as an adapted process, where ϕ_n denotes the number of shares held over the interval $[t_n, t_{n+1})$. In this setting, it is assumed that only the underlying stock and a risk-free asset can be traded. Consequently, the portfolio process satisfies

$$\begin{cases} X_{n+1} = X_n + \phi_n(S_{n+1} - S_n), & n = 0, 1, \dots, N-1, \\ X_N = x_0 + \sum_{n=0}^{N-1} \hat{\phi}_n(S_{n+1} - S_n). \end{cases}$$

Since the trading strategy can be represented as a measurable function of the state variables, namely $\phi_n \approx f(t_n, S_n, Y_n)$, the problem reduces to approximating an unknown function f defined on a compact subset of $\mathbb{R}^3$. By the universal approximation theorem, for any continuous function f on a compact

domain and for any $\varepsilon > 0$, there exists a feedforward neural network f_θ with at least one hidden layer, a finite number of neurons, and a non-linear activation function such that

$$\sup_{(t,S,Y)} |f(t, S, Y) - f_\theta(t, S, Y)| < \varepsilon.$$

Consequently, a neural network can act as a flexible parametric class capable of approximating the optimal trading strategy to arbitrary accuracy. This idea was used to create a standard feedforward neural network that takes in a 3D tensor input of shape $(M, N, 3)$. The inputs are the t_n, S_n , and Y_n and the output is our recommended trading strategy $\phi_0, \phi_1, \dots, \phi_{N-1}$ which the network determines after being trained and validated.

A rectified linear unit (ReLU) activation function, $\sigma(x) = \max(0, x)$, was employed in the hidden layers, while the output layer utilised a linear activation, i.e. $f_{\text{out}}(x) = x$. The use of ReLU alleviates the vanishing gradient problem by ensuring that, for $x > 0$, the derivative satisfies $\sigma'(x) = 1$, thereby preserving gradient flow during backpropagation. However, in sufficiently deep architectures, gradient degradation may still occur.

Batch normalisation was incorporated between successive layers to stabilise the distribution of intermediate activations. To further promote stable training, He initialisation was applied to the network weights. Specifically, for a layer with n_{in} input neurons, the weights are initialised as

$$w_{j,k}^{(i)} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}\left(0, \frac{2}{n_{i-1}}\right).$$

This choice ensures that the variance of the pre-activations remains approximately constant across layers, i.e.

$$\text{Var}(z^{(i)}) \approx \text{Var}(z^{(i-1)}),$$

which mitigates both vanishing and exploding gradient phenomena when using ReLU activations.

Table 1: FNN architecture with Batch Normalization and ReLU activations for question 3.2.

Layer	Type	Input shape	Output shape	Activation	Param #
Input	Input	$(N, 3)$	$(N, 3)$	None	0
1	Dense	$(N, 3)$	$(N, 100)$	Linear	300
2	BatchNorm	$(N, 100)$	$(N, 100)$	–	400
3	Activation	$(N, 100)$	$(N, 100)$	ReLU	0
4	Dense	$(N, 100)$	$(N, 100)$	Linear	10,000
5	BatchNorm	$(N, 100)$	$(N, 100)$	–	400
6	Activation	$(N, 100)$	$(N, 100)$	ReLU	0
7	Dense	$(N, 100)$	$(N, 100)$	Linear	10,000
8	BatchNorm	$(N, 100)$	$(N, 100)$	–	400
9	Activation	$(N, 100)$	$(N, 100)$	ReLU	0
Output	Dense	$(N, 100)$	$(N, 1)$	Linear	100
Trainable parameters:					21,001
Non-trainable parameters:					600
Total parameters:					21,601

Table 1 summarises the architecture of the neural network after compiling everything together and showcases how each layer feeds into the next.

To get started, input tensors had to be designed. These will have been used by the feedforward network in order to produce the desired strategies. Note `xtrain` has shape $(M, N, 3)$:

$$\text{xtrain}[m, n, :] = (t_n, S_n^{(m+1)}, Y_n^{(m+1)}),$$

This essentially contains an instant of the time, stock price, and variance. A big assumption made here is that since (S_t, Y_t) is Markovian, the whole stock price history does not need to be passed in. The label tensor is given by

$$y_{\text{train}}[m, n] = S_{n+1}^{(m+1)} - S_n^{(m+1)},$$

It follows that $y_{\text{train}} \in \mathbb{R}^{M \times N}$ acts as an instance of the simulated paths which we pass into the loss function. This is an unsupervised learning problem which means "true" realised y labels do not exist. The dataset is partitioned into training and validation sets using an 80%–20% split.

For the purposes of training the model a custom loss function needs to be created. This is because unlike traditional machine learning problems, there are no determined y labels to train against. The utility function is given by $U(x) = -e^{-\gamma x}$ as specified in the brief, then after defining $y_n = S_{n+1} - S_n$, it is possible to write $X_N = x_0 + \hat{\phi} \cdot y$. The artificial loss function is therefore expressed as:

$$\ell(y, \hat{\phi}) = -U(X_N) = \exp(-\gamma(x_0 + \hat{\phi} \cdot y)).$$

The model is then trained over 50 epochs where the aim is to minimise the loss, and the corresponding training and validation losses are recorded throughout the optimisation process in Figure 2. The training and validation loss steadily decrease indicating successful learning of a mapping of the state variables to the stock holding.

The plots presented in this subsection were generated using a model initially trained on a baseline set of parameters, as previously described. Individual parameters were then systematically varied to assess their impact on the model's predictions. Specifically, a loop structure was implemented to fix one parameter (e.g., variance) while varying another (e.g., stock holdings). This procedure was repeated across different scenarios, with the relevant parameter held constant and associated parameters adjusted accordingly.

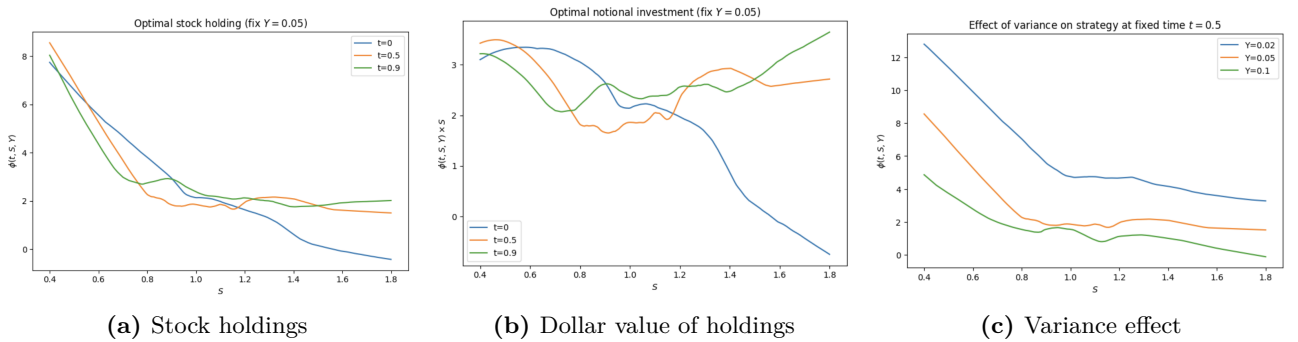


Figure 3: optimal stocks holding recommendations plus the effect variance has on the strategy

The optimal stock holding in Figure 3(a) exhibits a clear and economically intuitive dependence on the underlying asset price, time, and volatility level. In particular, the strategy is generally decreasing in S , reflecting a “buy-low, sell-high” behaviour that emerges from the exponential utility framework rather than simple drift exploitation. At higher prices, the marginal utility of wealth becomes more sensitive to downside risk, prompting the investor to reduce exposure or even take short positions as a form of hedging against adverse price movements. This effect is most pronounced at earlier times (e.g. $t = 0$), where the investor has greater flexibility to dynamically adjust positions, leading to more aggressive and sometimes negative allocations at high S . As time approaches maturity (e.g. $t = 0.9$),

the strategy becomes noticeably flatter and more conservative, reflecting the diminishing opportunity to recover from losses and the increasing importance of preserving terminal wealth. Additionally, the presence of mild irregularities or kinks in the learned policy can be attributed to a combination of neural network approximation error, Monte Carlo sampling noise, and the complexity of the stochastic problem.

When expressed in terms of notional investment in Figure 3(b), the strategy reveals a more non-linear structure due to the interaction between position size and asset price scaling. While the underlying allocation $\phi(t, S, Y)$ tends to decrease with S , multiplying by the price introduces a competing amplification effect, resulting in non-monotonic and sometimes irregular profiles. At earlier times, particularly $t = 0$, the notional exposure declines sharply and can become significantly negative for high values of S , indicating a strong hedging motive where the investor actively shorts the asset to mitigate downside risk associated with elevated price levels. In contrast, at later times such as $t = 0.9$, the notional investment tends to increase with S , as the strategy becomes less sensitive to price shifts and more focused on maintaining stable exposure in the face of limited remaining time. The increased irregularity observed in this representation is partly due to the amplification of approximation errors and simulation noise when scaling by S , but it also reflects the inherently nonlinear trade-off between risk and return embedded in the utility maximisation problem. Overall, this plot highlights how translating position sizes into monetary exposure can reveal additional structural features of the learned strategy.

The impact of the variance on the optimal strategy is both pronounced and consistent with theoretical expectations under risk-averse preferences. In Figure 3(c), as the instantaneous variance Y increases, the optimal allocation to the risky asset decreases significantly across all values of S , demonstrating the investor's sensitivity to heightened uncertainty. This inverse relationship aligns with classical results in portfolio optimisation, where optimal exposure is typically proportional to the inverse of variance, and is further reinforced by the exponential utility function, which heavily penalises downside risk. The effect is particularly strong at higher stock prices, where elevated variance compounds the potential for large losses, often leading to near-zero or even negative positions as the investor adopts a risk centric mindset. Although the Heston model implies that variance is mean-reverting, the learned strategy prioritises short-term risk considerations over long-term expectations, indicating that immediate uncertainty plays a dominant role in decision-making. Minor non-smooth features in the curves can again be attributed to numerical and approximation factors; however, the overall pattern captures the fundamental economic intuition that higher volatility discourages risk-taking and leads to more conservative portfolio allocations.

2.3 Extension: including the portfolio constraint

Attention is now directed towards the problem of utility maximisation within the same framework but now in the presence of a risk constraint C . This constraint enforces that the terminal value of the portfolio cannot be below a certain threshold, i.e. $X_T \geq C$

An intuitive way to incorporate the terminal constraint within the optimisation framework is to adopt a squared penalisation approach when the constraint is not satisfied [2]. Specifically, violations of the constraint are incorporated into the objective via an additional penalty term.

The resulting loss function is given by

$$\ell_C(y, \hat{\phi}) = \exp(-\gamma(x_0 + \hat{\phi} \cdot y)) + \lambda(C - (x_0 + \hat{\phi} \cdot y))_+^2,$$

where $(a)_+ = \max(a, 0)$ and $\lambda > 0$ is a penalty parameter.

This formulation corresponds to a constrained optimisation problem, in which the penalty term enforces the constraint $X_T \geq C$ by assigning non zero penalties to risk violations beyond a certain threshold. Consequently, the optimisation procedure favours admissible strategies that satisfy the constraint while still maximising expected utility.

The same feedforward neural network architecture, with identical parameters and hyperparameters, was employed, this time using the new loss function $\ell_C(y, \hat{\phi})$. The input tensors were constructed in the same manner as previously described. Furthermore, the training procedure retained the same number of epochs, and mini-batch normalisation was applied as inputs propagated through the network.

Following training, the unconstrained trading strategy is compared with the constrained strategy by means of graphical analysis, in order to assess the impact of incorporating risk control on the model's decision-making process in Figure 4. Note, the minimal impact of increasing λ above 50 on the constrained model.

From Figure 4, it is apparent that the constrained and unconstrained strategies diverge significantly for lower values of S_t . At the fixed time $t = 0.5$, the constrained strategy reflects the requirement that the portfolio must satisfy the terminal constraint $X_T \geq C$, which limits the extent to which the agent can take aggressive long positions. In particular, excessive exposure at low stock prices introduces substantial downside risk, as further adverse movements in S_t may lead to a violation of the terminal wealth constraint. As a result, the constrained strategy becomes highly conservative in this region, significantly reducing exposure to the risky asset and, in some cases, even approaching zero or slightly negative allocations. This behaviour highlights a shift in objective from pure return maximisation to capital preservation when the probability of constraint violation is high.

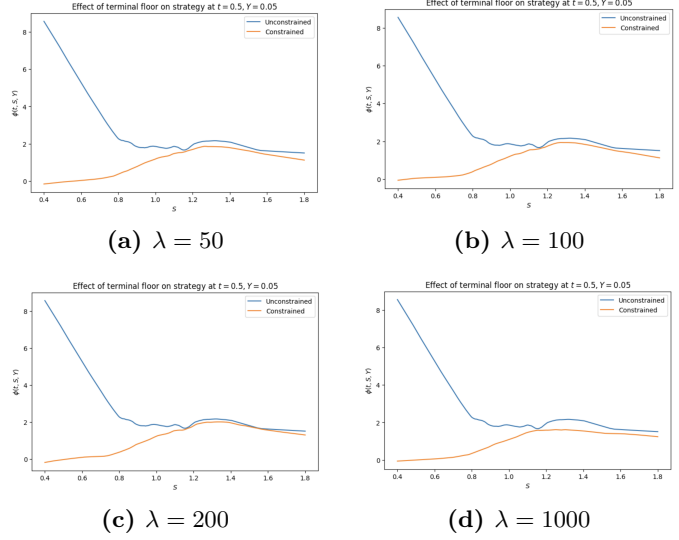


Figure 4: Effect of terminal floor with different penalty terms (λ)

By contrast, the unconstrained strategy exhibits markedly different behaviour, allowing for substantial long positions when the stock price is low, consistent with the expectation of potential upward movement and higher future returns. This aligns with standard financial intuition, whereby lower asset prices are viewed as favourable entry points for increasing exposure. However, unlike the constrained case, the unconstrained framework does not account for downside risk in a hard sense, and therefore permits aggressive risk-taking even in scenarios where large losses are possible. As S_t increases, the discrepancy between the two strategies diminishes, reflecting the fact that the constraint becomes less binding when the portfolio is less likely to fall below the threshold C . In this region, both strategies converge toward similar allocations, indicating that the constraint primarily affects behaviour in adverse states.

Overall, the figure illustrates that the introduction of a terminal wealth constraint induces a strongly state-dependent adjustment in the optimal strategy, effectively increasing the agent's risk aversion in regions where the constraint is most likely to bind. This results in a transition from aggressive, growth-oriented behaviour in the unconstrained case to a more cautious, risk-controlled approach under the constraint. Such behaviour is consistent with theoretical insights from constrained stochastic control problems, where the presence of a downside constraint alters the structure of the optimal policy by prioritising the avoidance of low-wealth outcomes over the pursuit of higher expected returns.

Figure 5 highlights that the portfolio constraint has been implemented effectively, since almost all of the portfolios satisfy the condition on terminal wealth. The improvement is evident from the decline in the violation probability from 0.18515 for the unconstrained portfolio to 0.0182 for the constrained portfolio. While the constrained portfolio clearly improves performance relative to the unconstrained case, the results also reveal that the model does not fully achieve the theoretical requirement that $P(X_T \geq C) = 1$. Instead, the simulated probability remains below 1, indicating that the model cannot

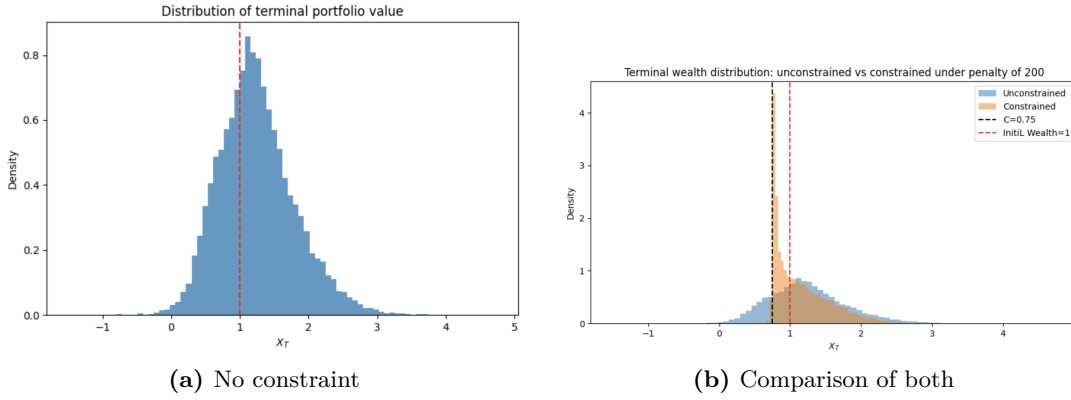


Figure 5: Distribution of terminal portfolio values with and without the constraint

enforce the terminal wealth constraint exactly because there is insufficient training data in the relevant range to learn this boundary condition accurately. Additionally, the simulated results may be subject to numerical approximation. Therefore, the findings should be interpreted as evidence of substantial improvement in constraint satisfaction rather than as perfect attainment of the target condition.

2.4 Conclusions

In conclusion, this section applied a feedforward neural network to solve two portfolio optimisation problems under the Heston stochastic volatility model. The unconstrained strategy behaved as expected since under exponential utility, stock holdings decreased with S_t , reflecting buy-low, sell-high dynamics, and reduced with higher Y_t as the investor became more risk averse under elevated uncertainty. Strategies also grew more conservative as maturity approached. Incorporating the terminal wealth constraint $X_T \geq 0.75$ via a squared penalty term significantly altered the strategy at low stock prices, reducing the breach probability from 0.185 to 0.018. Full theoretical satisfaction of $P(X_T \geq C) = 1$ was not achieved, likely due to insufficient training data near the constraint boundary. For future work, replacing the FNN with a recurrent architecture such as an LSTM could better capture temporal dependencies in the strategy. Systematic hyperparameter tuning, increased Monte Carlo paths, and importance sampling in the low-wealth region would all improve constraint satisfaction. Incorporating realistic market frictions such as transaction costs would also provide a more robust evaluation of the learned strategies.

3 Hedging and Pricing of an Exotic Option

3.1 Problem Statement

We again considered the Heston model and baseline parameters mentioned in Section 1. Additionally, it is assumed the investor can only trade the stock and risk-free asset with 0 interest rate and that the variance process Y_t was observable but non-tradable.

As outlined in [3], we moved onto a deep neural network approach to the pricing and then hedging of a down-and-out barrier put option with maturity (in years) $T = 1$, strike price $K = 1$ and barrier price $B = 0.7$. The payoff H_T of the option at maturity is:

$$H_T = (K - S_T)^+ \times \mathbf{1}_{\{L_T \geq B\}}, \quad L_T := \inf_{0 \leq u \leq T} S_u \quad (3-1)$$

Put simply, the option contract pays the typical European put payoff $\max(K - S_T, 0)$, provided that the stock price had not breached the barrier price before maturity, otherwise it is 'knocked out' and expires worthless.

It is assumed the investor is the option seller and has exponential utility $U(x) = -e^{-x}$ with an initial portfolio value $x_0 = 1$. The first objective was to determine the utility indifference sell price p^* , which is the premium for which the investor is indifferent between **a)** not selling the option or **b)** selling

the option while hedging optimally in the underlying market. The maximal expected utility for **a)** is denoted as V^0 and that for **b)** as $V^{sell}(p)$ where p is the premium. Then p^* is characterised by $V^{sell}(p^*) = V^0$.

The second objective was to investigate how p^* varies with parameters μ , η and ρ . Finally, the third objective was to determine how the optimal hedging strategy in **b)** depends on the state variables S_t , Y_t and the barrier level B . A feed-forward neural network (FNN) approach outlined in Section 3.2.3 was used to achieve these objectives.

3.2 Solution Methodology

In the present incomplete market setting, the utility-indifference pricing problem considered in this Section did not admit a closed-form solution. The main difficulties were at first that, the underlying Heston model is 2-factor and non-linear, meaning the optimal trading rule is not analytically tractable and second, the option payoff is path dependent due to the barrier feature, which complicated any direct dynamic programming or PDE based approaches. Therefore, a model-based deep learning approach was adopted, where Monte Carlo (MC) simulated market paths are used to train a FNN which approximates the optimal trading strategy.

3.2.1 Discretisation of the Continuous Time Control Problem

Although S_t and Y_t evolve in continuous time, the assignment specified that trading can only occur on a daily basis. Thus the same discrete time grid reformulation and portfolio value process X_t in Section 2.2 was adopted. As a result, the continuous hedging problem in t was transformed into a finite sequence of trading decisions in n which could be represented by a neural network acting on the market state each rebalancing date.

3.2.2 Monte Carlo Simulation of Heston Process

Many sample paths (100,000) of the state variables (S_t, Y_t) were generated under the Heston model using the Milstein methodology and expressions for S_{n+1} , Y_{n+1} and X_{n+1} outlined in Section 2.2. The barrier feature was captured by augmenting each path with the barrier survival indicator function I_n defined and recursively updated as:

$$I_n = \mathbb{1}_{\{\min_{0 \leq k \leq n} S_k \geq B\}}, \quad I_0 = 1, \quad I_{n+1} = I_n \times \mathbb{1}_{\{S_{n+1} \geq B\}} \quad (3-2)$$

This addition compressed the path dependence into a single additional state variable which meant that the network did not need the full price history to determine whether the claim had been knocked out, but only the current market state (S_n, Y_n) and survival status I_n . The terminal payoff could then be written in discrete form as:

$$H_T = (K - S_N)^+ \times I_N \quad (3-3)$$

The role of MC here was to first produce the training data required by the FNN. Each simulated path provided a sequence of observable states (t_n, S_n, Y_n) with the realised stock increments $S_{n+1} - S_n$, from which the terminal wealth (and thus utility) for any candidate strategy was computed. Secondly, MC approximated the expectations in the objective functions in Sections 3.2.3 & 3.2.4 which were analytically intractable.

3.2.3 Neural Network Approximations of Trading Policies

For simplification, the investor's optimal control was modelled as an adapted process, so at each trading date the stock holding depended only on the current available information. Under the Heston model, the Markovian state is given by (t, S_t, Y_t) , and (in the case of the option seller problem) I_t . Thus, the stock position could be appropriately parameterised by a FNN trained on the market paths simulated

in Section 3.2.2, as opposed to more complicated Recurrent Neural Network (RNN) implementations typically used for temporal data.

For the baseline trading policy without the option liability (denoted by superscript 0), the stock holding was approximated by:

$$\phi_n^0 \approx f_\theta(t_n, S_n, Y_n) \quad (3-4)$$

where f_θ is the function learned by the FNN and θ the set of parameter weights. The Python implementation of the FNN architecture used in this Section is summarised in Table 2. This is the same for both the models for ϕ_n^0 and ϕ_n^{sell} . The FNN is applied row-wise across the N rebalancing dates, meaning parameter weights are shared across time. The model takes input shape $(Batch\ Size, N, 3)$ and at each time step, the network sees the 3-dimensional feature vector (t, S_t, Y_t) . The network consists of 3 hidden layers with 64 neurons each and ReLU activation (for previously mentioned reasons in Section 2.2), followed by a 1-dimension linear output neuron/layer. The raw output is then passed through a 'squeeze' layer responsible for only reshaping and then a bounded output transformation (where z is the input to the last layer):

$$\phi = 10 \times \tanh\left(\frac{z}{10}\right) \quad (3-5)$$

which limits the FNN outputted stock position ϕ to lie in the interval $[-10, 10]$. Exactly 80% of the inputted MC simulated paths were used in training and the remaining 20% was used to compute the validation loss. Additionally, every FNN was trained over 50 epochs to ensure sufficient learning and monitor overtraining symptoms. Note that a small amount of hidden layers and neurons were used to keep runtimes tractable due to the computationally demanding nature of the utility indifference price problem involving training several FNNs per solve. To somewhat mitigate this, the Python code was configured to run via the GPU to greatly reduce run times, with further guidance being provided in the submitted notebook. Time constraints meant additional improvements like batch normalisation, hyperparameter tuning, activation function investigations etc. were not implemented and left for future work.

Table 2: FNN architecture for trading models. There are 3 hidden layers with 64 neurons.

Layer	Type	Input shape	Output shape	Activation	Parameter no
Input	Input	$(N, 3)$	$(N, 3)$	None	0
1	Dense	$(N, 3)$	$(N, 64)$	ReLU	256
2	Dense	$(N, 64)$	$(N, 64)$	ReLU	4160
3	Dense	$(N, 64)$	$(N, 64)$	ReLU	4160
Output	Dense	$(N, 64)$	$(N, 1)$	Linear	65
5	Lambda squeeze	$(N, 1)$	(N)	None	0
6	Lambda output transform	(N)	(N)	Scaled tanh	0
Total trainable parameters:					8641

Under the approximation for ϕ_n^0 , the terminal wealth computation becomes:

$$X_T^0(\theta) = x_0 + \sum_{n=0}^{N-1} f_\theta(t_n, S_n, Y_n)(S_{n+1} - S_n) \quad (3-6)$$

and the baseline (without option selling) optimisation problem is to obtain V^0 in equation 3-7 which is the expected maximum utility of the terminal wealth. Due to the FNN approximation f_θ , we instead optimise over θ by using $\mathcal{L}_{base}(\theta)$ as the loss function during training. Note, that tensorflow requires a loss function to **minimise** over, hence the sign change compared to the equation. In the Python implementation, the expectation is replaced by the empirical average of $X_T^0(\theta)$ calculated using the FNN weights, over the MC sample.

$$V^0 = \sup_{\phi} \mathbb{E}[-e^{-X_T}] \approx \sup_{\theta} \mathbb{E}[-e^{-X_T^0(\theta)}] = -\mathcal{L}_{base}(\theta) \quad (3-7)$$

A direct FNN approximation of the option seller's hedge ϕ_n^{sell} was determined to be unsatisfactory in the knocked-out region where economically, after the barrier is breached, the option liability disappears and the put seller's trading policy should collapse back to the baseline policy. If the seller's ϕ_n^{sell} policy had to be learnt entirely from scratch, the FNN had to deduce this structural property indirectly, which led to unstable and economically nonsensical behaviour. To this end, the ϕ_n^{sell} policy was instead posed as:

$$\phi_n^{sell} = \phi_n^0 + I_n \Delta_{\psi}(t_n, S_n, Y_n) \quad (3-8)$$

Notice the use of ϕ_n^0 , the already trained baseline trading policy and Δ_{ψ} , which is the FNN trained for the seller's hedge problem. This Δ_{ψ} can be thought of as the 'hedging correction' from the baseline policy. This representation ensured that when the option is already knocked out and $I_n = 0$, then automatically $\phi_n^{sell} = \phi_n^0$, which is the correct economic behaviour. In addition to capturing the barrier economics, this approach is also advantageous as it was numerically easier to train an FNN for a comparatively small correction to the baseline trading policy than one for an entirely new policy from scratch. Notice, that I_n was not included explicitly as an input feature but instead is handled externally in the path simulation and payoff evaluation. Thus, the use of I_n did not increase the number of network parameters.

In this formulation, the seller's terminal wealth is defined in equation 3-9 and similar to $\mathcal{L}_{base}(\theta)$, the hedge correction network is trained by minimising $\mathcal{L}_{sell}(\psi, p)$ defined in equation 3-10 as the loss function. Again, the expectation was approximated by the MC average over the simulate paths.

$$X_T^{sell}(\psi, p) = x_0 + p - H_T + \sum_{n=0}^{N-1} (\phi_n^0 + I_n \Delta_{\psi}(t_n, S_n, Y_n))(S_{n+1} - S_n) \quad (3-9)$$

$$V^{sell}(p) = \sup_{\phi} \mathbb{E}[-e^{-X_T}] \approx \sup_{\Psi} \mathbb{E}[-e^{-X_T^{sell}(\psi, p)}] = -\mathcal{L}_{sell}(\psi, p) \quad (3-10)$$

3.2.4 Utility Indifference Price Computation

The utility indifference price p^* is such that $V^{sell}(p^*) = V^0$. Numerically, p^* was solved via a bisection search. An initial wide interval $[p_{low}, p_{high}] = [0, 1]$ was chosen so that the root was guaranteed to lie within. The FNN for the baseline case needed only to be trained once to obtain V^0 which is independent of p .

At each iteration, the midpoint and candidate premium $p_{mid} = (p_{low} + p_{high})/2$ was computed. The seller correction FNN was trained and corresponding utility $V^{sell}(p_{mid})$ was estimated using the validation paths. The interval was then updated based on conditions involving p_{mid} . If $V^{sell}(p_{mid}) < V^0$, p_{mid} was too small and the lower bound is updated to $p_{low} = p_{mid}$. Otherwise, p is too big and the upper bound is updated to $p_{high} = p_{mid}$. Iterations of this procedure progressively narrowed the interval containing the root.

The algorithm was terminated when the absolute difference $|V^{sell}(p_{mid}) - V^0|$ falls below 10^{-3} , which was the smallest computationally feasible tolerance given that each evaluation of $V^{sell}(p_{mid})$ requires training an FNN. Note, bisection was a simple and appropriate solution to this problem due to the monotonic dependence of $V^{sell}(p)$ on p , which reduced the pricing problem to a 1-dimensional root finding problem.

3.2.5 Extraction of Hedge Profiles

After FNN training, the hedge correction result for the option seller is defined by rearranging equation 3-8 to yield equation 3-11. This object was appropriate to analyse since it isolated the incremental effect of the option liability from the investor's underlying optimal portfolio choice.

$$\phi_n^{sell} - \phi_n^0 = I_n \Delta_\psi(t_n, S_n, Y_n) \quad (3-11)$$

3.3 Computational Results

3.3.1 Model Training and Validation Loss

Before reporting prices and hedge profiles, it was important to verify the FNN optimisation itself behaved sensibly. For the baseline parameter set, the FNNs were trained on the MC simulated Heston paths using the utility based loss functions mentioned in Section 3.2.3. The corresponding training and validation losses were recorded over the 50 epochs and plotted on figure 6 for the baseline and final seller correction model respectively.

In the baseline problem, both training and validation losses initially decline sharply and then steadily before flattening out, indicating the FNN is learning a non-trivial mapping from the state variables (t, S_t, Y_t) to the stock holding ϕ^0 . The absence of any explosive behaviour suggests the optimisation is numerically stable. A similar pattern can be observed for the option seller correction model. Although, the seller losses improve more gradually, both curves still show a clear downward trend. This is consistent with the correction term representing a smaller adjustment to the already optimised baseline strategy.

Note, that the validation loss being above the training loss is not concerning as the loss was evaluated on MC path samples. Consequently, small differences between training and validation losses can arise because one sample was simply 'easier' than the other. Overall, these training losses support that the network have learnt meaningful properties. Similar qualitative behaviour was observed across the other FNN fits used throughout the analysis in Sections 3.3.2&3.3.3.



Figure 6: Training and validation loss for the baseline portfolio (a) and seller correction (b) network under the baseline parameter set

3.3.2 Utility Indifference Price Results

The utility indifference price was computed using the methodology discussed in Section 3.2. For the **baseline** parameter set, the estimated utility indifference price p^* was **0.04688**. For the following parameter sensitivity study of μ , η and ρ , 1 parameter at a time was varied at 10 discrete points within a defined interval, whilst holding the other baseline parameters constant.

Dependence on μ

Figure 7(a) shows that p^* remained unchanged at roughly $p^* = 0.04688$ as the drift parameter μ varies over the range $[0.0, 2.0]$. Economically, it was expected that a higher drift would mean that S_T is less likely to finish below K and expire worthless, thus reducing the value of the put liability and so the premium p^* demanded.

The unexpected result could arise for several reasons. Firstly, over the simulated 1 year horizon in the chosen parameter range, the effect of μ may be too weak compared to variance and barrier effects. In this stochastic volatility section, the distributional impact of the volatility dynamics can dominate moderate drift changes, especially when pricing is driven by downside path events as opposed to the mean of the terminal portfolio wealth distribution alone. Secondly, since bisection was used to compute p^* numerically, if the true variation of p^* with respect to μ was of order less than 10^{-3} , then the coarse tolerance would mask genuine drift effects and produce repeated identical estimates. Third, the residual MC and FNN optimisation noise may be comparable in magnitude to the drift sensitivity. The safest interpretation was that within the numerical resolution, p^* appeared insensitive to μ and any dependence was smaller than the bisection tolerance.

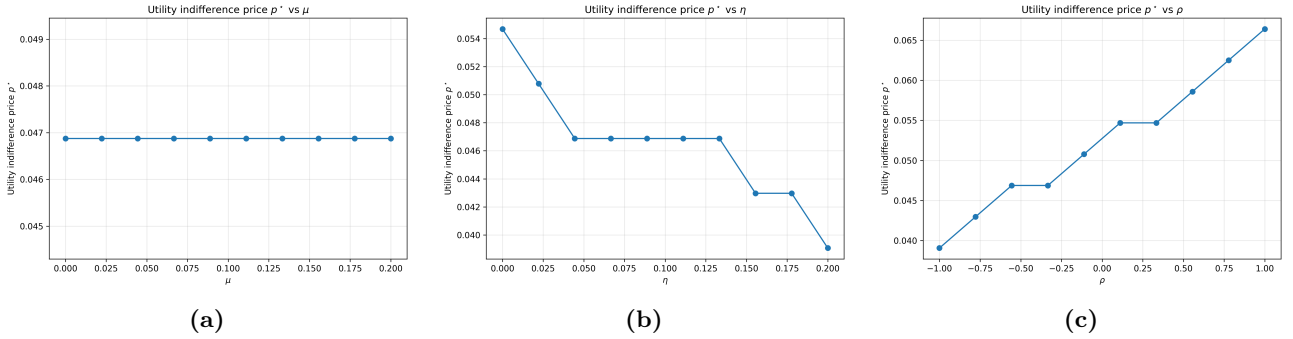


Figure 7: Sensitivity plots of p^* versus μ (a), η (b) and ρ (c), holding all other baseline parameters constant

Dependence on η

Figure 7(b) shows that there is an overall decrease in p^* as the 'volatility of volatility' η increases in the range $[0.0, 2.0]$, despite the plateaus. There are two opposing economic intuitions. One could argue higher uncertainty in the variance process would generate fatter downside tails and more severe in-the-money outcomes. This coupled with an exponential utility which punishes losses more than equivalent gains would increase p^* . Conversely, the same mechanism also increases the probability of the option crossing B and being knocked out, removing the payoff liability and thus reducing p^* . The numerical results suggested at the current calibration, this barrier knockout effect dominated.

The plateaus visible for intermediate η values most likely reflected the previously mentioned numerical effects off the bisection tolerance and MC/FNN optimisation noise making the sensitivity curve appear piecewise constant over some intervals. However, the overall downward trend was sufficiently pronounced to support the conclusion that increased η reduced the effective value of the barrier liability.

Dependence on ρ

Figure 7(c) shows a clear upward trend of p^* when ρ increases in the range $[-1.0, 1.0]$. Again there are 2 opposing economic intuitions. On one hand, one could expect a more negative correlation to strengthen the 'leverage' effect, (where stock price declines are accompanied by increases in volatility), which increases downside risk and makes put option liabilities greater, thus increasing p^* . Conversely, stronger downside stock movements raise the probability of crossing B and reducing p^* as explained earlier. Note positive ρ values are unlikely in real-life practice and were investigated for interest. Again, the numerical evidence suggested for the chosen parameter regime, the barrier knockout effect dominated the increased downside volatility effect.

It is worth noting, the monotone pattern was inferred from a finite grid and numerically estimated prices which were again susceptible to MC/FNN optimisation noise and the bisection algorithm tolerance which could smooth out/hide small local irregularities. Nonetheless, the ρ trend was sufficiently strong to not be attributed to a numerical coincidence.

3.3.3 Hedging Strategy Results

The methodology described in Section 3.2 was used to train the seller's FNN correction model on the baseline parameters and obtain the hedge correction $\phi_t^{sell} - \phi_0^t$. Then the effects of S_t , Y_t and B on the hedging correction strategy were examined. In order to isolate the dependence on the individual state variable, cross sections were constructed either as functions of S_t for fixed variance levels Y_t or as functions of Y_t for fixed stock price levels S_t . The effect of the barrier level B was studied by repeating a similar analysis for several different B levels while holding the remaining parameters fixed. The correction was evaluated at several representative times $t \in \{0, 0.5, 0.9\}$. Note that like the previous sensitivity analysis, the computational results in this subsection were susceptible to being artificially smoothed by bisection tolerance, MC/FNN optimisation noise or discretisation error from the cross-section tests.

Effect of S_t

The hedge correction profiles as functions of S_t are shown for variance levels $Y_t = 0.02$, $Y_t = 0.05$ and $Y_t = 0.20$ in figure 8. The correction is exactly 0 at and below the barrier, consistent with the intention behind the approach of the correction model. Therefore, once the option is knocked out, the seller's trading policy collapses to the baseline policy. Additionally, at an intermediate region above the barrier, the hedge correction becomes negative. This is economically consistent with a short-put position providing positive delta (stock price increase) exposure and so the hedge model should provide a counteracting negative delta signal (sell stock shares). For sufficiently large S_t , the hedge correction returns to 0, reflecting the fact that put option becomes increasingly out-the-money and therefore the derivative exposure weakens, causing the needed hedge to also increase towards 0.

Notably, a pronounced positive spike appeared immediately above the barrier where, near the knock-out B level, a small downward S_t move both simultaneously increases put moneyness, and also probability of hitting the barrier, meaning the correction hedge becomes highly non-linear. It was seen that the barrier effects dominated in Section 3.3.2, so perhaps the positive hedge correction was the FNN model trying to aggressively offload any accumulated negative stock hedging position as the FNN predicts that the option is likely to hit the barrier. Additionally, investigations revealed only 2.9% of the simulated MC S_t paths were in the range $0.7 \leq S_t \leq 0.8$, meaning it was likely that the FNN had insufficient data in this region to learn the above barrier dynamics correctly.

Comparing the 3 variance levels shows that larger Y_t amplifies the magnitude of the positive barrier region spike and increases the S_t range over which the hedge correction remains noticeably different from 0. In particular, when Y_t is high, the transition after the barrier into negative hedging corrections shifts towards higher S_t values. Economically this makes sense as higher variance increases both the previously mentioned knock-out risk and also the probability of higher $S_t > K$ paths still finishing in-the-money where $S_T < K$. Sensibly, as t increases towards 1, the hedging profiles become more aggressive. This reflected the reduced time available to adjust positions and unwind risk. Additionally, the distribution of $S_T|S_t$ became more concentrated, meaning the range of future price movements narrowed, leading the model to adopt more decisive trading decisions.

Effect of Y_t

Figure 9 shows the hedge correction as a function of Y_t for different fixed stock price levels $S_t = 0.7$ (barrier region), $S_t = 1.0$ (at-the-money region) and $S_t = 1.5$ (deep out-the-money region).

At $S_t = 0.7$, the hedge correction is positive for all 3 time slices and increases strongly with Y_t , indicating that when the stock is at the barrier, greater Y_t increases the hedge correction. Economically,

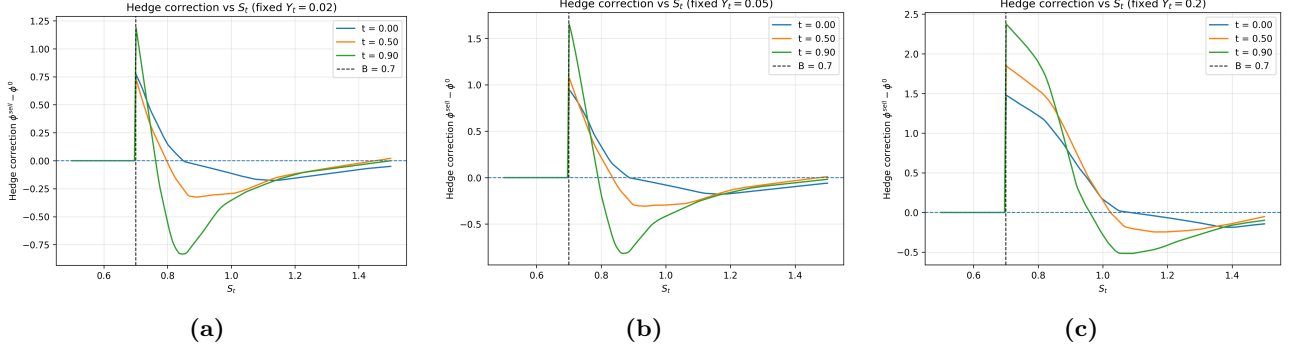


Figure 8: Cross-section plots of $\phi_t^{sell} - \phi_0^t$ versus S_t for $Y_t = 0.02$ (a), $Y_t = 0.05$ (b) and $Y_t = 0.20$ (c), holding all baseline parameters constant for $t = 0.0, 0.5, 0.9$

this is consistent with higher volatility near the barrier increasing the probability of the option being knocked out causing the previously mentioned dynamics and hedge correction increase. Here higher t makes the hedge correction model trade more aggressively.

At $S_t = 1.0$, the correction is negative for all time slices and its magnitude increases with both Y_t and t , consistent with higher variance raising the probability of an in-the-money finish and prompting more aggressive hedging. The anomalous $t = 0$ curve sitting above the later time slices at elevated Y_t is most plausibly a training data artefact: paths with high instantaneous variance at $t = 0$ are sparse in the Monte Carlo sample, so the FNN correction $\Delta\psi$ generalises poorly in this region of the state space and the resulting curve ordering reflects approximation error rather than genuine economic structure.

In contrast, at $S_t = 1.5$, the hedge correction is small in magnitude for all times and remains close to 0. Economically, when $S_t \gg K$ the put is deeply out-the-money and substantial increases in Y_t have limited effects on the seller's liability. The small variation reflects that there still exists some finite probability of large downward movements (which increase with Y_t) especially at earlier values of t . Notably, again no intuition could be gathered for why the $t = 0.5$ line was above the others.

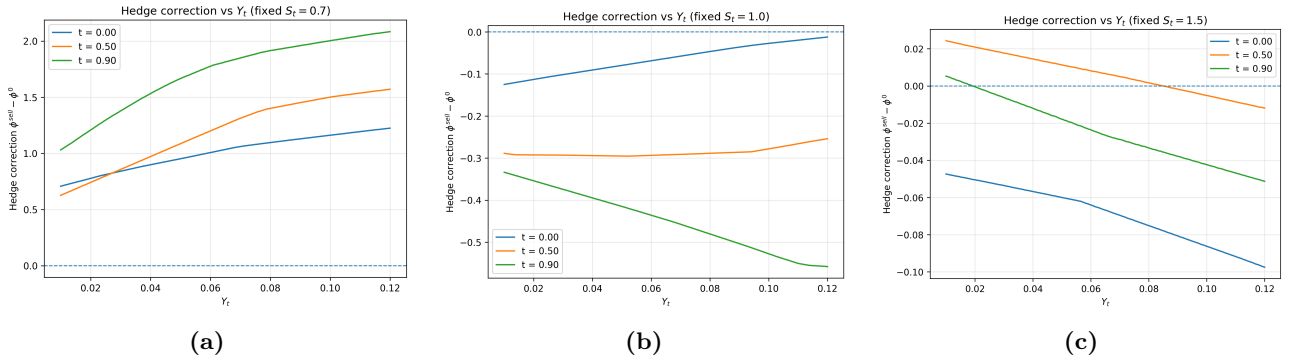


Figure 9: Cross-section plots of $\phi_t^{sell} - \phi_0^t$ versus Y_t for $S_t = 0.70$ (a), $S_t = 1.00$ (b) and $S_t = 1.50$ (c), holding all baseline parameters constant for $t = 0.0, 0.5, 0.9$

Effect of B

Figure 10 shows the hedge correction profiles against S_t for variance level $Y_t = 0.05$ under 3 barrier levels $B = 0.4$, $B = 0.7$ and $B = 1.4$.

Note figure 10(b) is identical to figure 8(b) and repeated for easy comparison purposes. When the barrier is low and $B = 0.4$ the lines look identical to the $B = 0.7$ case it was trained on in the region $S_t \geq 0.7$. However, below $S_t = 0.7$, the predictions are extrapolations which terminate at $S_t \approx 0.5$. Additionally, one would expect the region of predicted positive hedge correction to transition to

negative hedge correction at much lower S_t . This can be attributed to the insufficient training data in these regions, causing erroneous predictions.

When the barrier level is $B = 1.4$, the hedging correction appears extremely fragile. The correction is much smaller in overall magnitude away from the barrier than for lower B and the entire hedging profile appears shifted towards the right compared to $B = 0.7$. Intuitively, with a very high barrier level, most states below $B = 1.4$ correspond to a knocked-out claim so meaningful hedge adjustments only occur in a narrow region just above the barrier. Even these appear to move towards 0 for higher S_t .

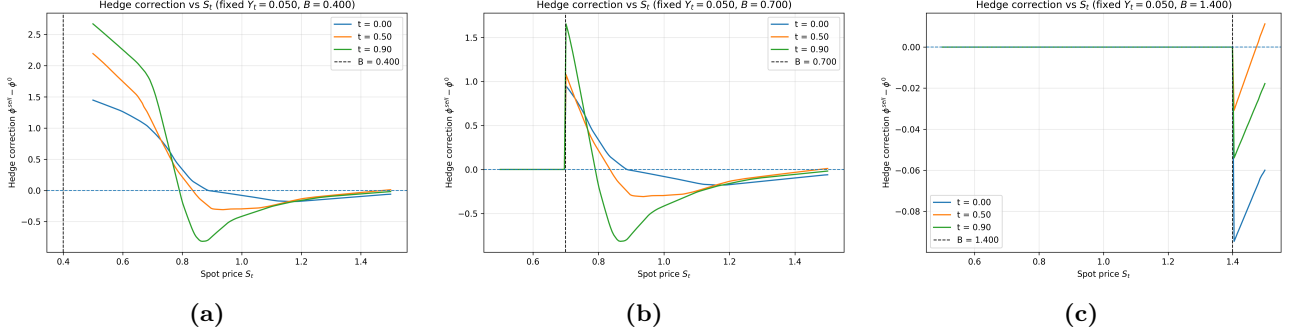


Figure 10: Cross-section plots of $\phi_t^{sell} - \phi_0^t$ versus S_t for $B = 0.40$ (a), $B = 0.70$ (b) and $B = 1.40$ (c), holding all baseline parameters constant for $t = 0.0, 0.5, 0.9$

3.4 Conclusions

This section studied the utility indifference pricing p^* and hedging of a 'down-and-out' barrier put under a discretised version of the Heston stochastic volatility model using a FNN approach. For the baseline parameter set, $p^* = 0.04688$. The pricing sensitivity analysis revealed that within the numerical resolution of the bisection implementation, p^* was largely insensitive to μ , decreased with η and increased with ρ . The latter 2 observations revealed that for the barrier claim, knock out effects can dominate the standard vanilla-put economic intuitions.

The hedging correction results showed a strong state dependence. Specifically, the correction disappeared once the option was knocked out, became negative over intermediate regions which the short-put exposure dominated and returned towards 0 at high S_t values where the option was sufficiently far out-the-money. A notable feature was the pronounced positive spike immediately above the barrier level which reflected the non-linear interaction between the put being in-the-money and knock-out risk. Additionally, the dependence of hedge correction on Y_t and B further confirmed that the barrier feature significantly alters the structure of the optimal hedging correction behaviour.

Results should be interpreted with some caution nonetheless. Each price was estimated through a nested procedure involving MC simulation, FNN optimisation and bisection, meaning the sensitivity and cross section plots were subject to numerical noise, tolerance error and discretisation effects. For example, small plateaus in price curves may have reflected the bisection tolerance rather than true sensitivity. Moreover, some hedge profile irregularities just above the barrier level may have reflected the scarcity of simulated training data in that region, causing the FNN model to learn poorly there.

In terms of future work, the numerical accuracy could be improved by tightening the bisection tolerance, increasing the number of MC paths and carrying out a systematic hyper-parameter tuning process for the FNN models (eg. learning rate). Another opportunity is exploring richer neural network architectures and batch normalisation techniques which may improve the stability of the learned hedge correction profiles, particularly close to the barrier. Finally, the implemented model could be extended with more realistic market frictions like transaction costs to assess how robust the conclusions we drew are, beyond the idealised frictionless setting of this project.

4 Group Member Task Breakdown

Divij

- Creation of latex template
- Code implementation of question 3.3
- Report write-up of question 3.3
- Attempt at incorporating X as a state in FNN model in question 3.2

Syed

- Report write up of 3.2
- Attempted to implement an RNN/LSTM based solution to 3.3 but did not manage to get a solution

Omar

- Code implementation for part 3.2. One notebook using Euler simulations and another with Milstein.
- Researched how to implement the penalty constraint for part 3.2.
- Helped Syed with the write-up for part 3.2

Max

- Wrote the introduction (Section 1), including the presentation of the Heston model framework and baseline parameter setup.
- Assisted with code formatting and organisation for Section 2 (Portfolio Optimisation).
- General proofreading and improvements to the report throughout.

Bibliography

- [1] Numerical Analysis of Euler and Milstein Schemes with Applications to Multi-Asset Heston Models and Exotic Option Pricing; 2025. Accessed: 28 March 2026. Available from: https://files.globalpresshub.com/wp-content/uploads/2025/09/Revised-ms_AJPAM_2116_v1.pdf.
- [2] Tibshirani R. Convex Optimization Lecture 16: Penalty Methods; 2013. Carnegie Mellon University, 10-725 Convex Optimization. Accessed: 28 March 2026. Available from: <https://www.stat.cmu.edu/~ryantibs/convexopt-F13/scribes/lec16.pdf>.
- [3] Tse A. MATH0062 Mathematics and Statistics of Algorithmic Trading Group Assignment 2025 - 2026; 2026. Available from: https://ucl-eu-west-2-moodle-sitedata.s3.eu-west-2.amazonaws.com/1a/62/1a62f255c83d1cd430a1f1172c3cbd52a6abae6e?response-content-disposition=inline%3B%20filename%3D%22MATH0062%20Group%20Assignment%202025-26.pdf%22&response-content-type=application%2Fpdf&X-Amz-Content-Sha256=UNSIGNED-PAYLOAD&X-Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Credential=AKIA47YHZF637GKGWUJC%2F20260405%2Feu-west-2%2Fs3%2Faws4_request&X-Amz-Date=20260405T101413Z&X-Amz-SignedHeaders=host&X-Amz-Expires=21587&X-Amz-Signature=5112d07a044da971cd8125db8ea4f3fdca01e2e9a3a8c457cb0ee15abb590461.